



O pipeline que o time consegue ler.

Sparquet descreve engenharia de dados como **um JSON que o Spark executa**. O Studio abre esse mesmo arquivo como um canvas — e mostra de onde vem cada coluna.

PySpark

Delta · Iceberg · Kafka · JDBC

Catálogo e linhagem

Open source · Apache 2.0

O PROBLEMA

Todo pipeline vira código que só uma pessoa entende.

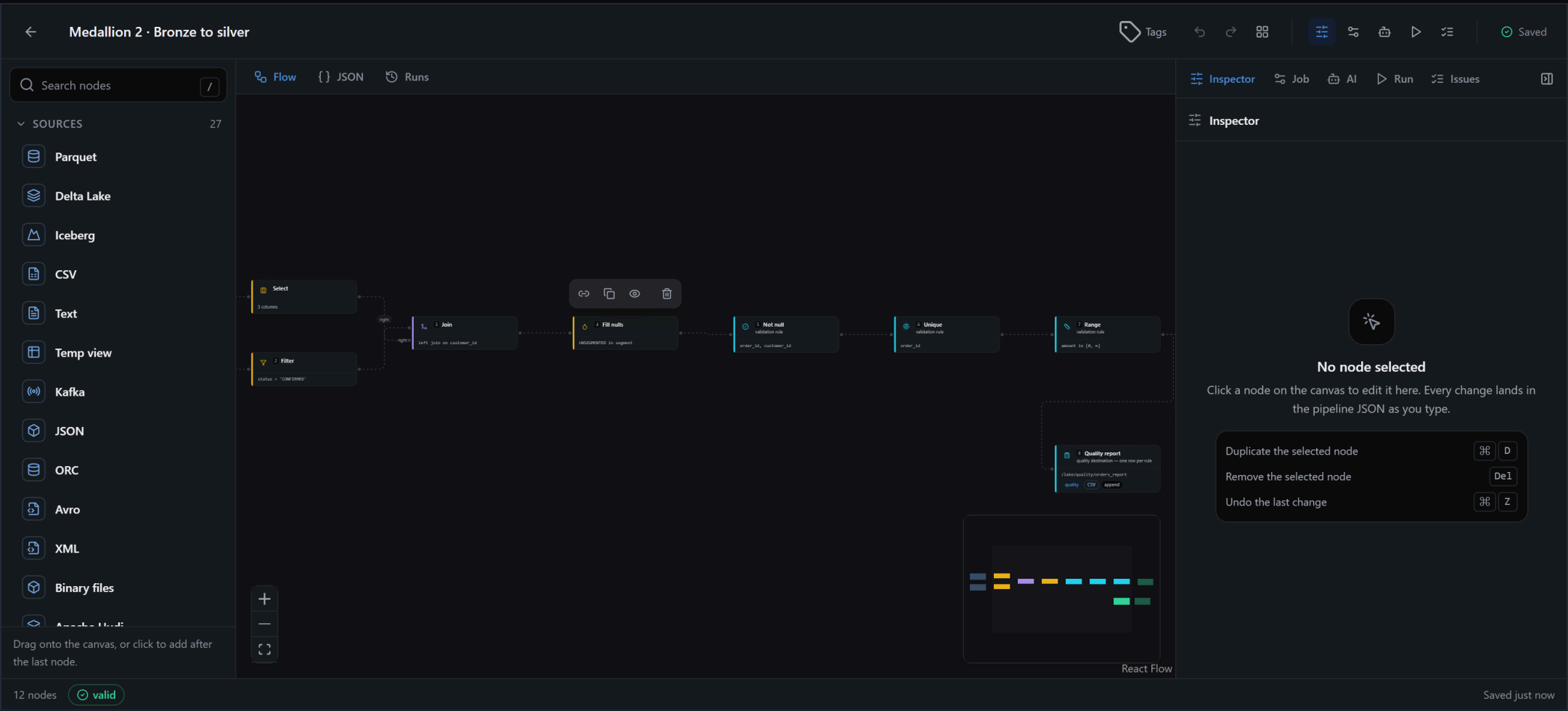
- O notebook cresce, a **regra de negócio some** entre um join e um withColumn.
- Ninguém sabe **quem lê a tabela** que você está prestes a mudar.
- Revisar em pull request é ler diff de código, não ler o fluxo do dado.
- O catálogo corporativo depende de alguém lembrar de cadastrar.

Sparquet inverte a ordem: o pipeline é **um documento**. Versionável, revisável, executável em qualquer Spark — local, Databricks, EMR, Dataproc ou Synapse.

DESENHE

O canvas é o pipeline.

Arraste fonte, transformação, regra de qualidade e destino. Cada nó tem formulário próprio, com validação e exemplos — **não é diagrama de documentação**, é o pipeline que roda.



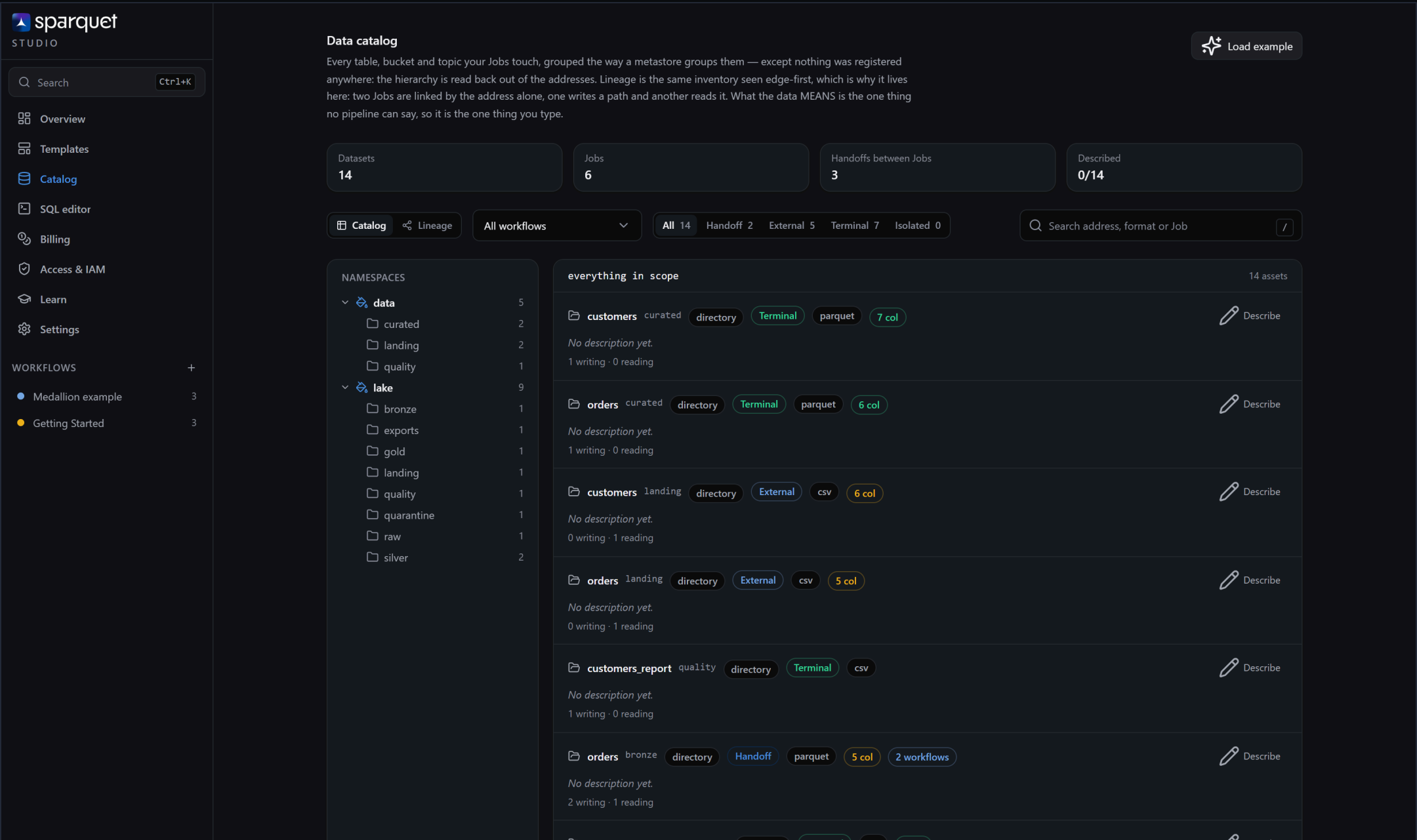
Job real: leitura em Parquet, join com clientes, preenchimento de nulos, três regras de qualidade e quarentena das linhas inválidas.

O mesmo arquivo que o Spark executa.

Sparquet Studio

Um catálogo que ninguém precisou cadastrar.

Cada tabela, bucket e tópico que seus Jobs tocam, agrupado como um metastore agrupa — só que **a hierarquia é lida dos próprios endereços**. Delta e Iceberg aparecem como tabela, arquivos como diretório, Kafka como tópico.

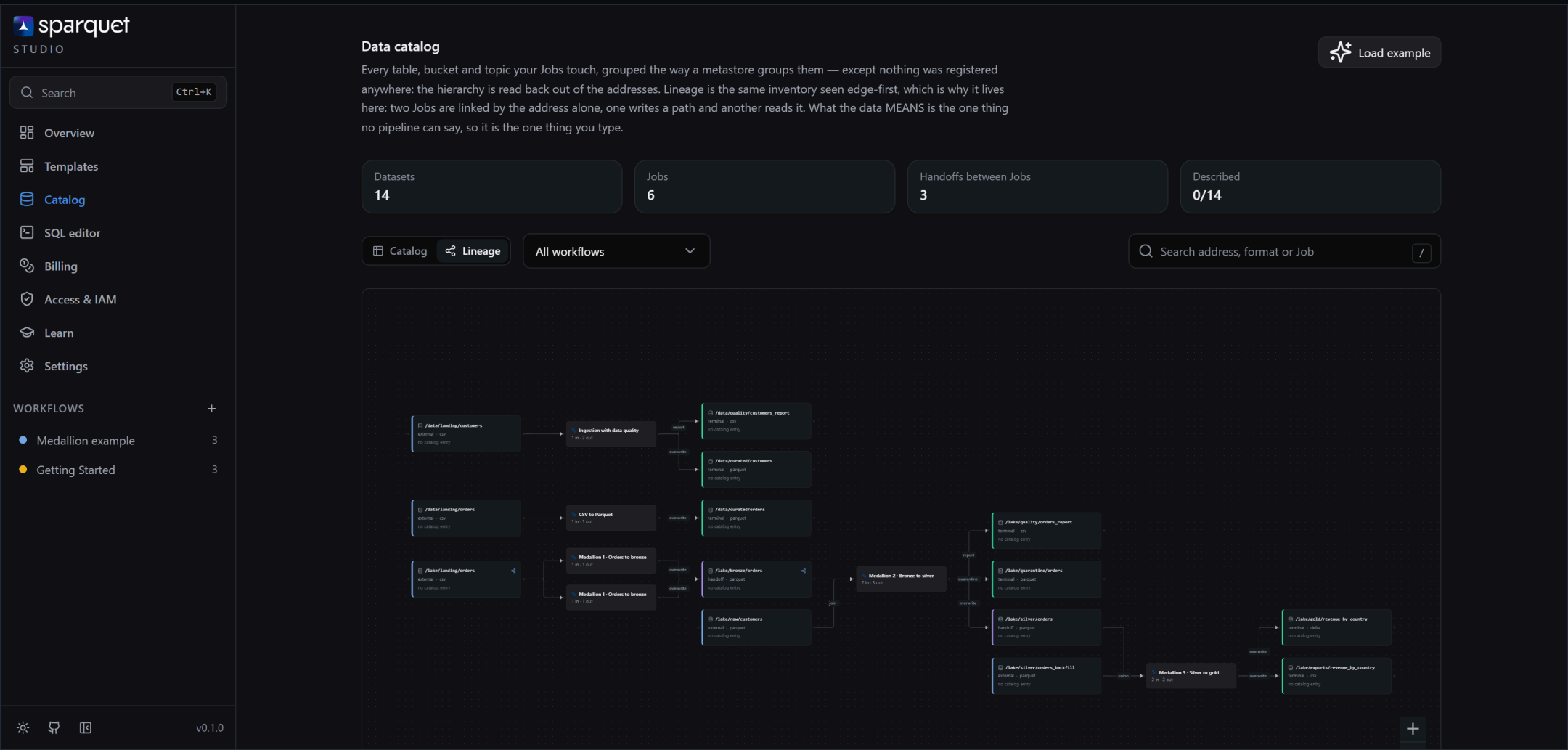


Schema derivado do canvas, com o grau de confiança dito na tela — e o botão que lê o schema real no storage para comparar.

LINHAGEM

Do landing ao gold, sem ninguém desenhar.

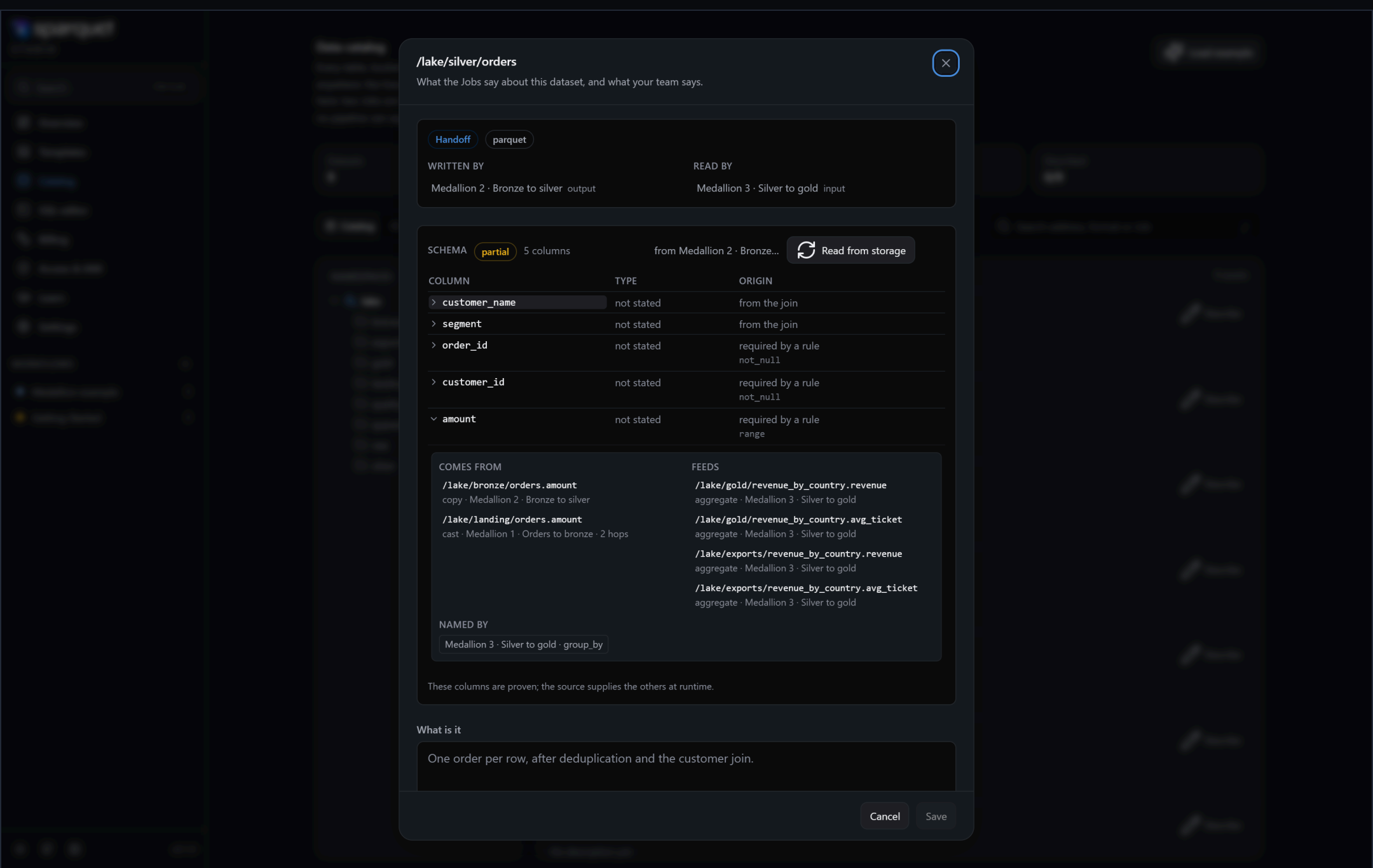
Dois Jobs se ligam **pelo endereço**: um escreve o caminho, outro lê. A linhagem é o mesmo inventário visto pela aresta — inclusive os destinos que existem só para relatório de qualidade e quarentena.



Arquitetura medalhão de exemplo: 14 datasets, 6 Jobs, 3 entregas de um Job para outro.

“O que quebra se eu mexer nesta coluna?”

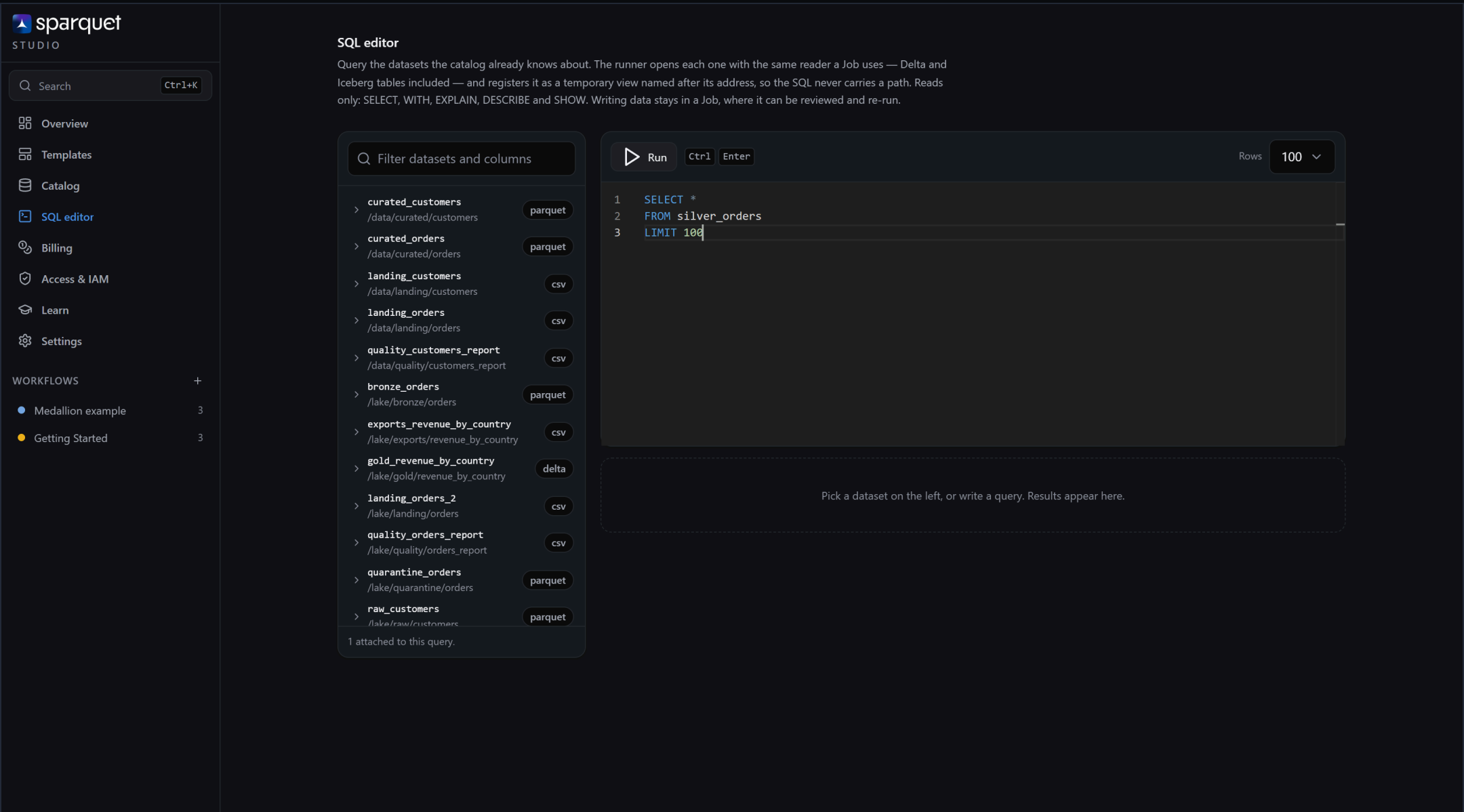
Linhagem no nível da coluna, atravessando Jobs: **de onde vem** este número, **o que ele alimenta** lá na frente, e **quem apenas o cita** num filtro, numa chave de join ou numa regra de qualidade.



Um rename quebra os dois; uma troca de tipo quebra só o primeiro. Por isso o Studio separa as duas coisas.

E quando a pergunta é mais rápida em SQL.

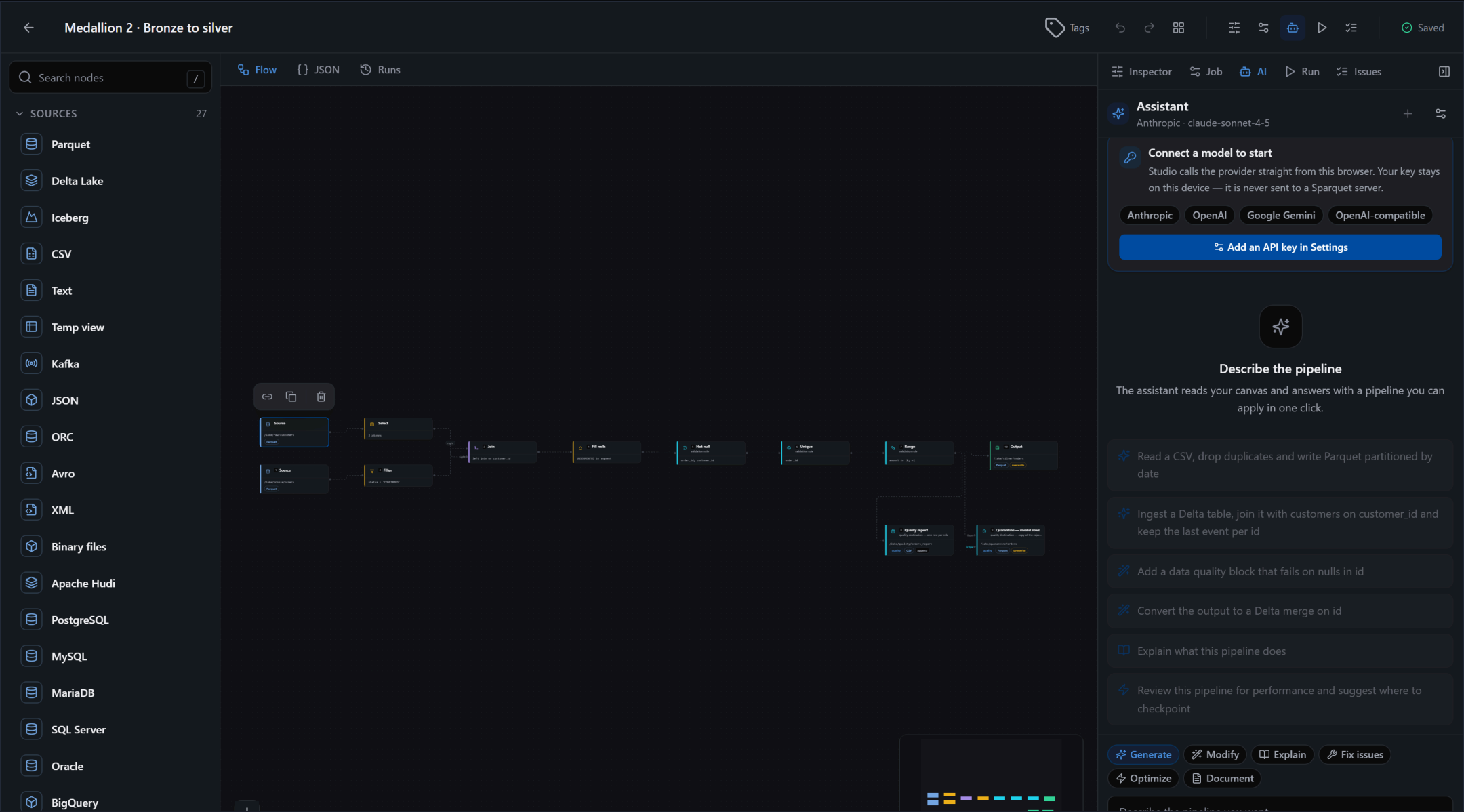
O catálogo já sabe o endereço e o formato de cada base, então **consultar é só dar um nome**. O runner abre cada tabela com o mesmo leitor que um Job usa — Delta e Iceberg inclusive — e registra como view temporária. Somente leitura: SELECT, WITH, EXPLAIN, DESCRIBE e SHOW.



Nenhum caminho na consulta, nenhuma credencial no navegador: quem lê o disco é o runner, na sua máquina.

IA que conhece a sua paleta.

Descreva o pipeline e receba um JSON válido, pronto para aplicar em um clique — gerar, modificar, explicar, otimizar ou corrigir os problemas do linter. O prompt sai do mesmo catálogo que desenha os formulários, então **a IA não inventa parâmetro que não existe.**

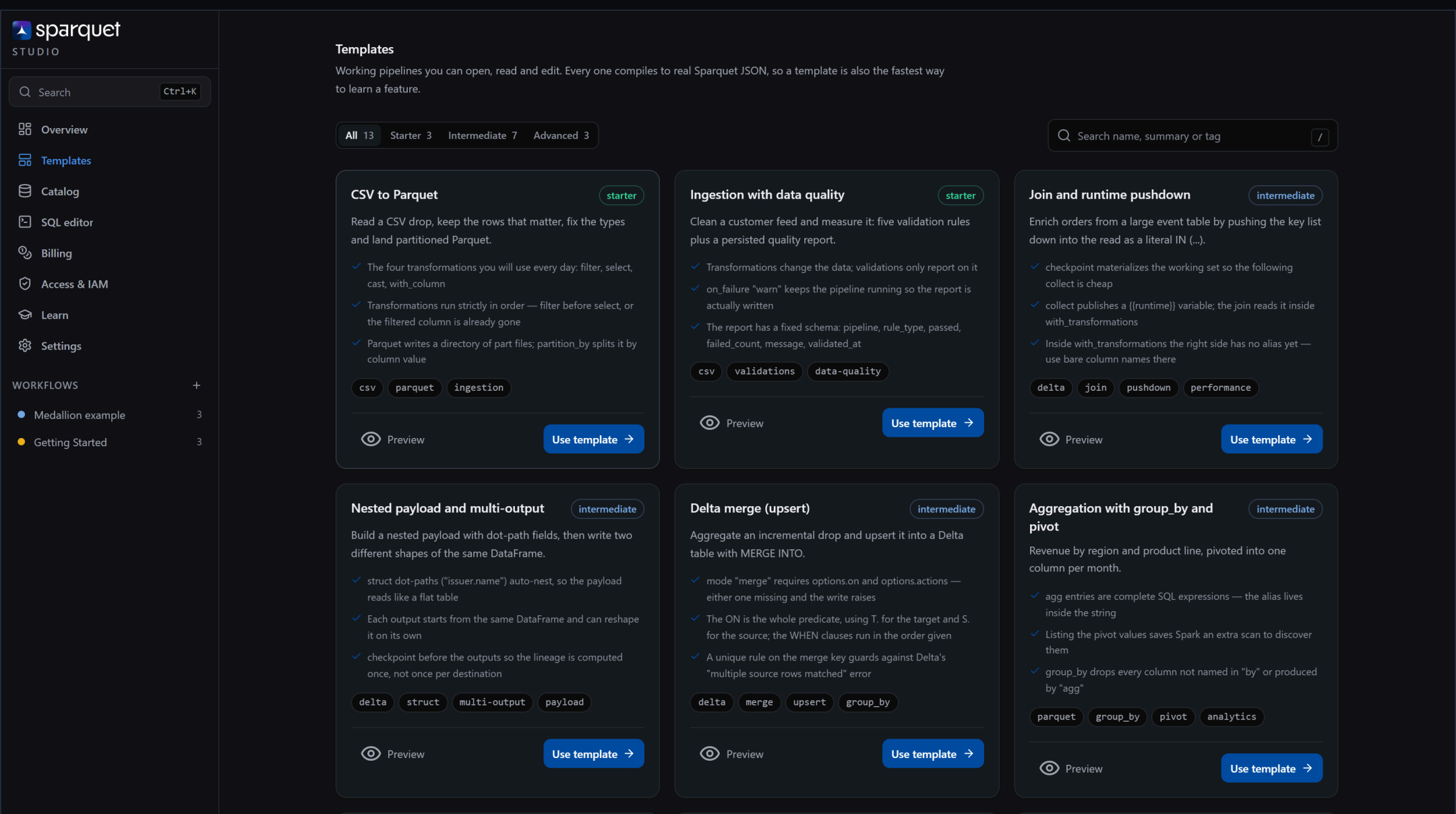


Anthropic, OpenAI, Gemini ou qualquer endpoint compatível. A chave fica no seu navegador e nunca vai para um servidor Sparquet.

ONDE RODA

Na sua máquina. Sem conta, sem telemetria.

O Studio é um app de navegador que guarda tudo localmente. Quando você quer executar de verdade, um runner opcional em **127.0.0.1** sobe o Spark, transmite o log em tempo real e devolve o relatório de qualidade — com token e papéis por ação.



Comece por um template pronto: ingestão com data quality, medalhão de três Jobs, CSV para Parquet.

Bateria inclusa.

27

fontes e destinos: Parquet, Delta, Iceberg, CSV, JSON, ORC, Avro, XML, Hudi, Kafka, JDBC, BigQuery, Snowflake, Mongo, Cassandra e mais

19

transformações — filter, select, cast, join com broadcast, group_by, union, sql, struct, checkpoint, collect

10

regras de qualidade com política fail/warn/skip, relatório por regra e quarentena linha a linha

Delta & Iceberg

MERGE de upsert e time travel do Delta, declarados no mesmo JSON

{param} • {{var}}

parâmetros de template e variáveis de runtime que viram pushdown no próximo read

4 nuvens

Databricks, EMR, Dataproc e Synapse detectados sozinhos — e local, para desenvolver

O motor de qualidade é a biblioteca **sparquet-cola**, publicada à parte e usável sozinha.

COMECE AGORA

Dois comandos.

```
$ pip install sparquet  
$ sparquet pipeline.json # roda em qualquer Spark
```

```
$ cd sparquet-studio && npm install && npm run dev  
http://localhost:5273
```

Site e documentação em 3 idiomas · sparquet.dev

Código, roadmap e issues · github.com/VictorPasqualini/sparquet

Qualidade de dados como lib · pip install sparquet-cola

Licença Apache 2.0. Sem conta, sem servidor obrigatório, sem telemetria.